

The Ultimate C++ & Git Master Guide

A complete, end-to-end curriculum for mastering C++, Version Control, and Automation.

Part 1: The 10-Module C++ Curriculum Overview

- **Module 1: Git & Tooling** (Version control, branching, remotes, rebase, cherry-pick, github).
- **Module 2: The Core Language** (Syntax, primitive types, control flow, loops, functions).
- **Module 3: Memory Architecture** (Pointers, heap/stack, dynamic memory, core Data Structures).
- **Module 4: OOP & Design** (Classes, polymorphism, inheritance, virtual functions, design patterns).
- **Module 5: STL & Modern C++** (Smart pointers, containers, algorithms, C++11 to C++20 features).
- **Module 6: Templates & Generic Programming** (Metaprogramming, function/class templates).
- **Module 7: Build Systems & Testing** (CMake, GDB debugging, Unit testing with GTest).
- **Module 8: System-Level Programming** (Concurrency, multithreading, mutexes, async, file I/O).
- **Module 9: Graphics & OpenGL** (Shaders, buffers, matrix math, real-time rendering pipelines).
- **Module 10: Advanced Capstones** (3D Engine, IoT Telemetry Server, Vision Processing Pipeline).

Part 2: Git Fundamentals & The Three Trees

Git tracks changes over time safely. To master Git, you must understand where your code lives at any moment:

- **Working Directory:** The actual files on your computer disk that you are currently editing.
- **Staging Area (Index):** A rough draft space. You `git add` files here when you plan to commit them.
- **Local Repository (.git):** The permanent timeline. You `git commit` files here.

Part 3: The Exact Step-by-Step Git Workflow

Step 1: Configuration

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Step 2: Initialization

```
mkdir my-project
cd my-project
git init
```

Step 3: Staging & Committing

```
git status
git add main.cpp
git commit -m "Initial commit"
```

Step 4: Ignoring Files

```
echo "*.exe" > .gitignore
git add .gitignore
git commit -m "Add gitignore"
```

Step 5: Standard Branching & Merging

```
git switch -c feature-math    # Create and switch to branch
# ... make changes ...
git add .
git commit -m "Add math logic"
git switch main              # Go back to main
git merge feature-math       # Bring changes into main
```

Step 6: Pushing to GitHub

```
git remote add origin https://github.com/user/repo.git
git branch -M main
git push -u origin main
```

Part 4: Advanced Git Concepts & Explanations

As requested, here are the detailed explanations and commands for advanced Git operations.

1. Branching Strategies & Rebase

While `merge` combines two branches by creating a new "merge commit", `rebase` rewrites history by moving your feature branch to the very tip of the main branch, creating a perfectly linear, clean history.

```
git checkout feature-branch
git rebase main
```

Warning: Never rebase commits that have already been pushed to a public repository!

2. Resolving Merge Conflicts

When Git cannot automatically merge or rebase, it inserts conflict markers into your code. You must manually fix them.

```
<<<<<<< HEAD
std::cout << "Current branch code";
=====
std::cout << "Incoming branch code";
>>>>>>> feature-branch
```

Delete the markers, keep the desired code, save, and run `git add .` followed by `git commit`.

3. Cherry-Pick

Sometimes you don't want to merge an entire branch, but you need one specific commit from it. **Cherry-picking** copies a specific commit from one branch and applies it to your current branch.

```
git cherry-pick <commit-hash>
```

4. Bisect (Debugging History)

If you discover a bug but don't know which commit caused it, `git bisect` performs a binary search through your commit history to find the exact commit that broke the code.

```
git bisect start
git bisect bad           # Mark current commit as broken
git bisect good <commit-hash> # Give it an older commit that worked
# Git will checkout a commit in the middle. Test it, then type:
git bisect good # or git bisect bad
# Git will keep narrowing it down until it finds the culprit.
git bisect reset       # End the debugging session
```

5. Stash

Used to temporarily shelf messy, uncommitted changes so you can switch branches cleanly.

```
git stash          # Hides changes
git stash pop      # Brings them back
```

Part 5: Remote Repositories Overview

Managing remote servers (like GitHub, GitLab):

- `git clone <url>` : Downloads a full repository to your machine.
- `git fetch` : Downloads all new history from the remote, but does **not** merge it into your working files. Safe way to check for updates.
- `git pull origin main` : Fetches changes and immediately **merges** them into your local branch.
- `git push origin main` : Uploads your local commits to the remote.
- `git remote -v` : Lists active remote connections.
- `git remote set-url origin <new-url>` : Changes where your local repo is pointing.

Part 6: Complete Command Reference

Modifying & Undoing History

- `git reset --soft HEAD~1` : Undoes the last commit; keeps files in the staging area.
- `git reset --hard HEAD~1` : **DANGEROUS**. Completely destroys the last commit and all uncommitted changes.
- `git restore <file>` : Discards unstaged changes in a specific file.
- `git rm <file>` : Removes a file from tracking and deletes it from disk.
- `git mv <old> <new>` : Renames a file cleanly in Git.

Windows Batch Scripting (.bat) Automation

Common commands for automation: `echo`, `@echo off`, `cls`, `pause`, `title`, `color`, `set`, `VAR=value`, `if`, `goto`, `call`, `start`, `cd`, `dir`, `mkdir`, `rmdir`, `del`, `copy`, `move`, `ren`, `xcopy`, `robocopy`, `tasklist`, `taskkill`, `ping`, `ipconfig`, `shutdown`, `exit`.

Example Auto-Push BAT Script:

Save the following in a file named `auto_push.bat` inside your project directory to automate backups.

```
@echo off
echo Building and Syncing to GitHub...
git add .
git commit -m "Auto commit backup"
git push origin main
echo Done!
pause
```